

II. Nondeterministic Actions

The agent **may not know the next state** after taking an action in a nondeterministic environment.

II. Nondeterministic Actions

The agent *may not know the next state* after taking an action in a nondeterministic environment.

A *belief state* is a set of physical states believed to be possible by the agent.

II. Nondeterministic Actions

The agent *may not know the next state* after taking an action in a nondeterministic environment.

A *belief state* is a set of physical states believed to be possible by the agent.

Problem solution: a *conditional plan*.

II. Nondeterministic Actions

The agent *may not know the next state* after taking an action in a nondeterministic environment.

A *belief state* is a set of physical states believed to be possible by the agent.

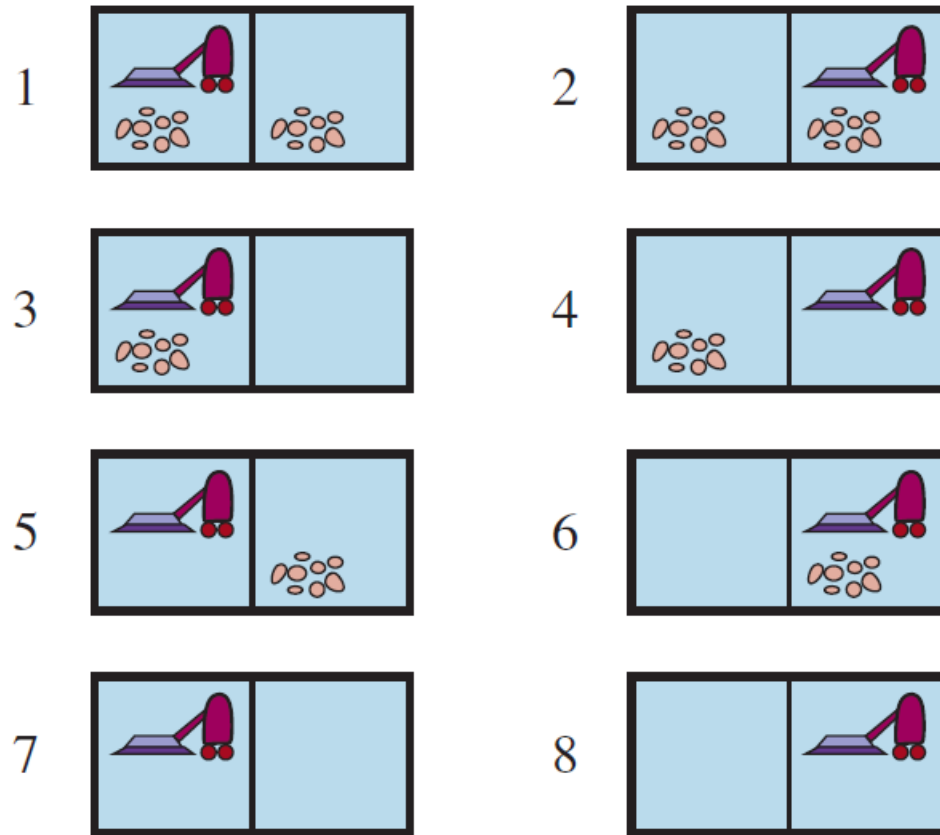
Problem solution: a *conditional plan*.



To specify what to do depending on what percepts the agent receives while executing the plan.

Perfect Vacuum World

Fully observable, deterministic, and completely known

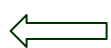


8 possible states

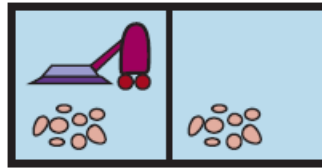
Perfect Vacuum World

Fully observable, deterministic, and completely known

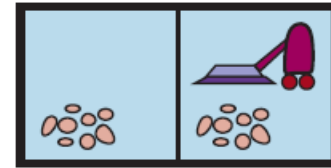
Solution: *Suck, Right, Suck* ←



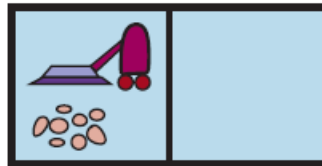
1



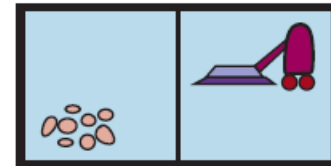
2



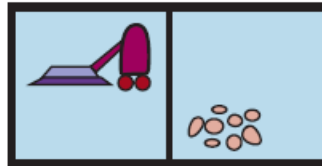
3



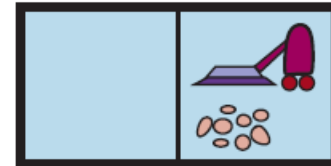
4



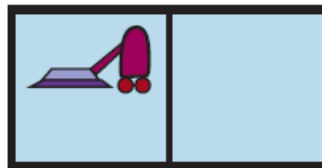
5



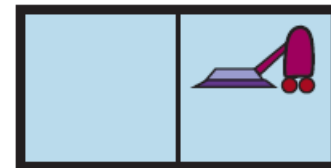
6



7



8

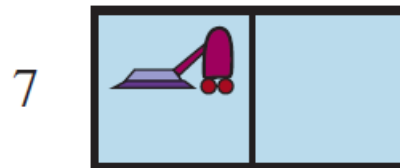
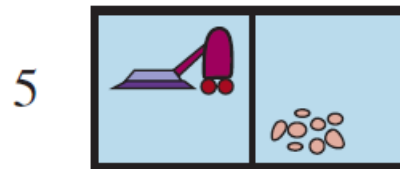
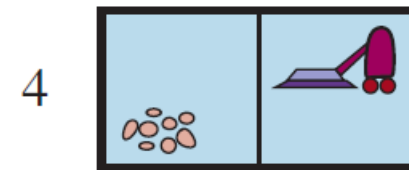
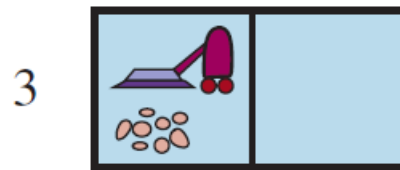
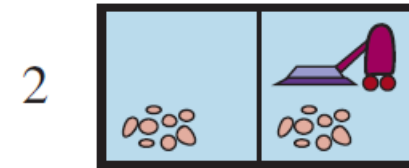
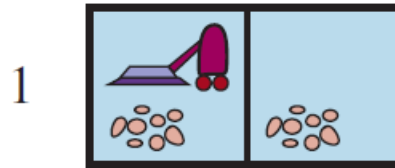


8 possible states

Erratic Vacuum World

Suck:

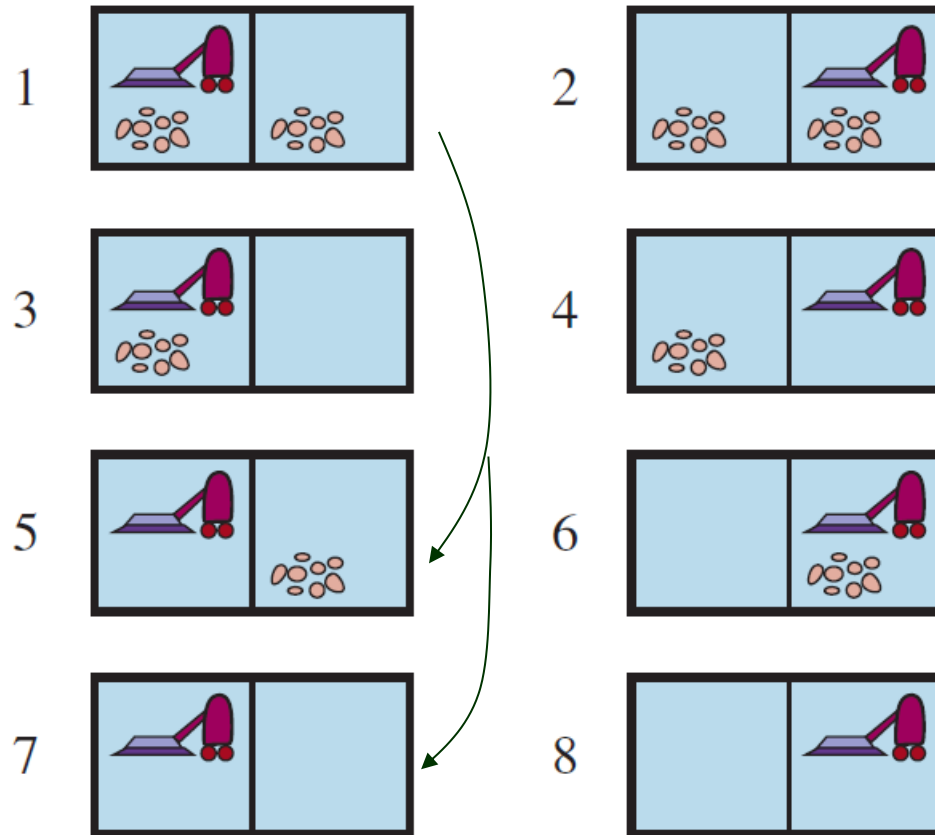
- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square



Erratic Vacuum World

Suck:

- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square

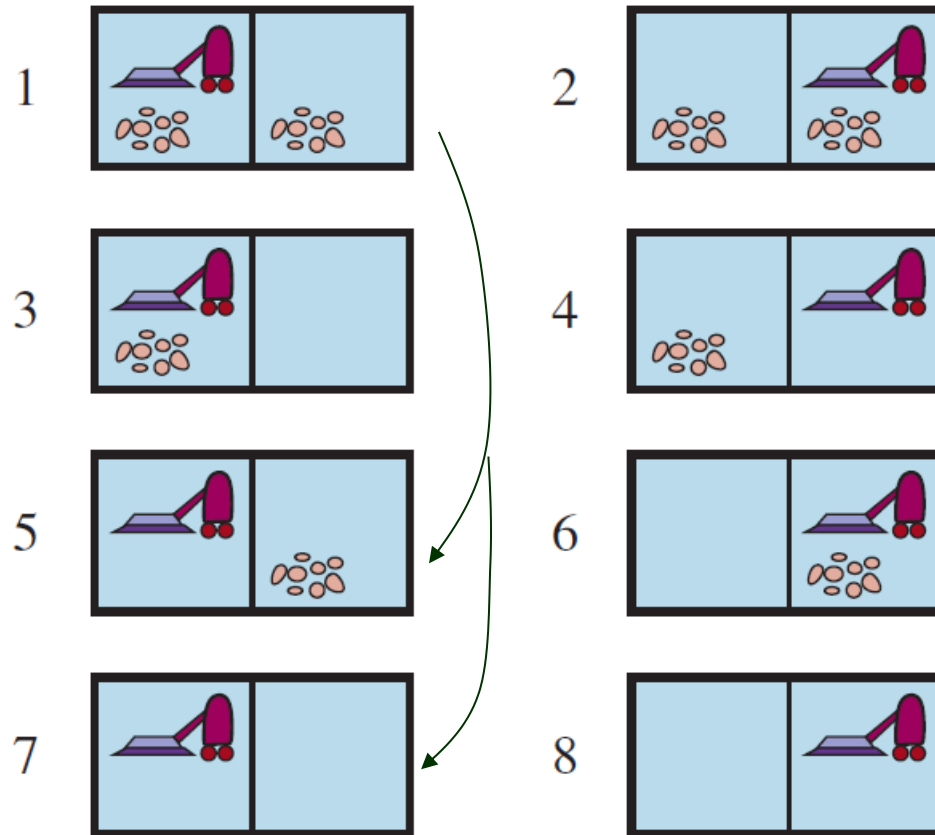


Either state 5 or 7 after applied to state 1:

Erratic Vacuum World

Suck:

- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square



Either state 5 or 7 after applied to state 1:

$$\text{RESULTS}(1, \text{Suck}) = \{5, 7\}$$

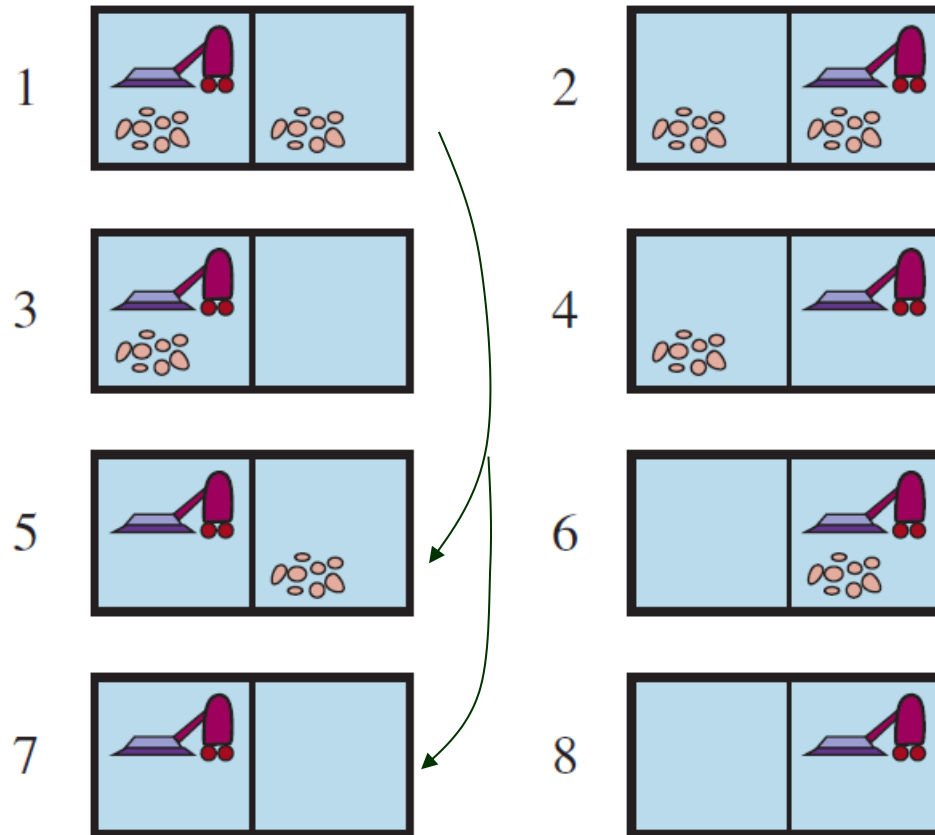
Erratic Vacuum World

Suck:

- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square
- Applied to a clean square
 - ◆ sometimes deposit dirt on the carpet

Either state 5 or 7 after applied to state 1:

$\text{RESULTS}(1, \text{Suck}) = \{5, 7\}$



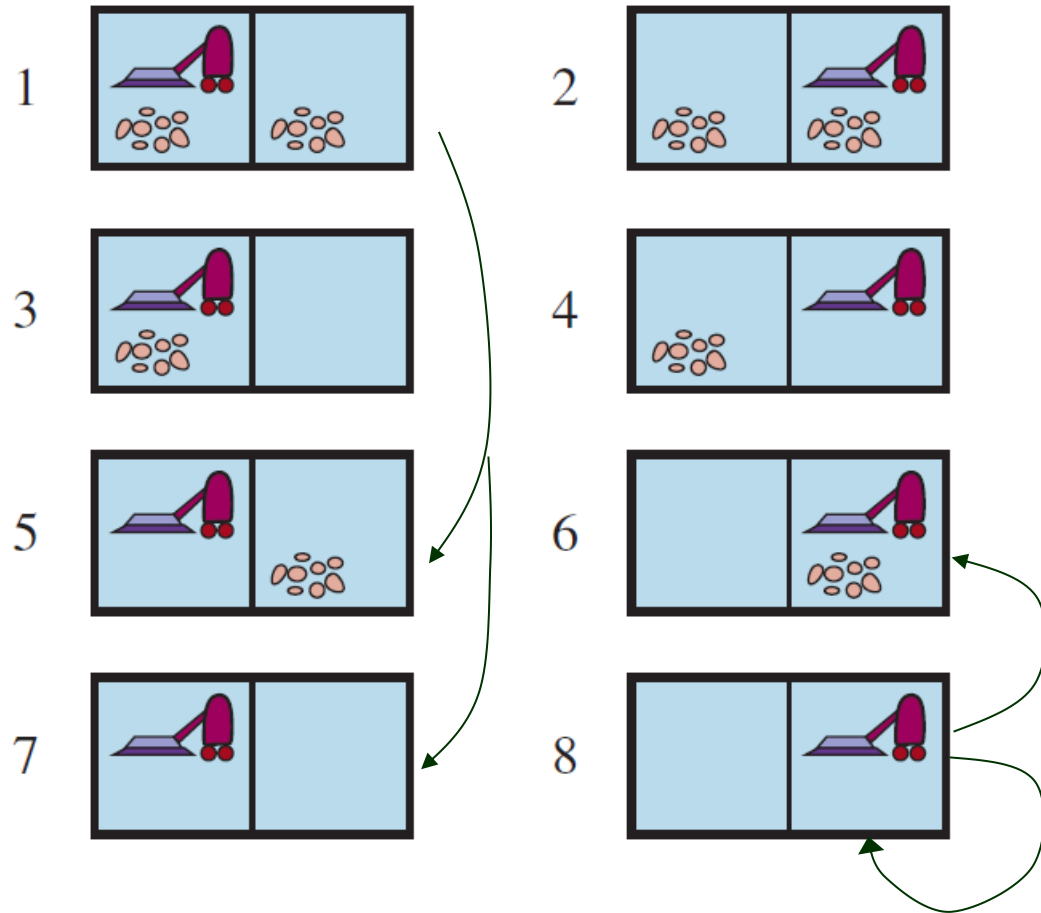
Erratic Vacuum World

Suck:

- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square
- Applied to a clean square
 - ◆ sometimes deposit dirt on the carpet

Either state 5 or 7 after applied to state 1:

$\text{RESULTS}(1, \text{Suck}) = \{5, 7\}$



Erratic Vacuum World

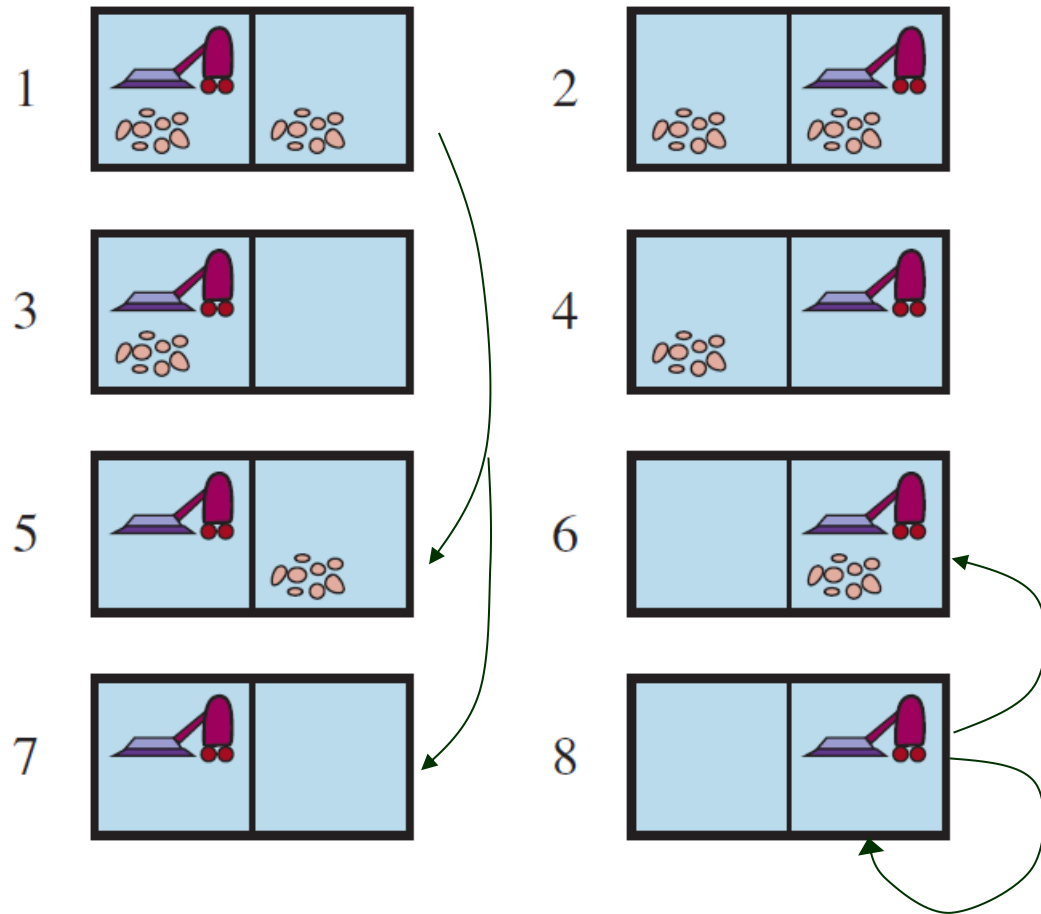
Suck:

- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square
- Applied to a clean square
 - ◆ sometimes deposit dirt on the carpet

Either state 5 or 7 after applied to state 1:

$\text{RESULTS}(1, \text{Suck}) = \{5, 7\}$

$\text{RESULTS}(8, \text{Suck}) = \{6, 8\}$

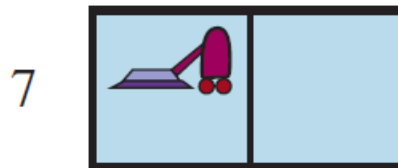
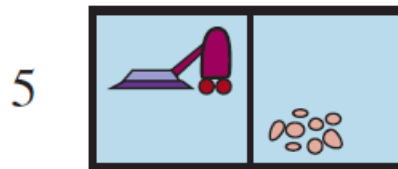
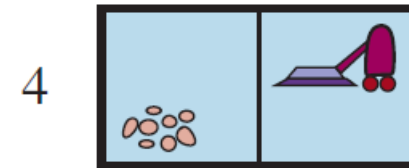
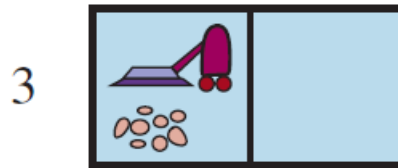
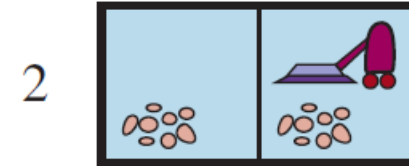
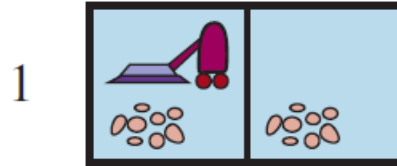


Still Solvable?

Suck:

- Applied to a dirty square
 - ♦ clean the square
 - ♦ sometimes also clean up dirt in an adjacent square

No single sequence of actions solves the problem.



Still Solvable?

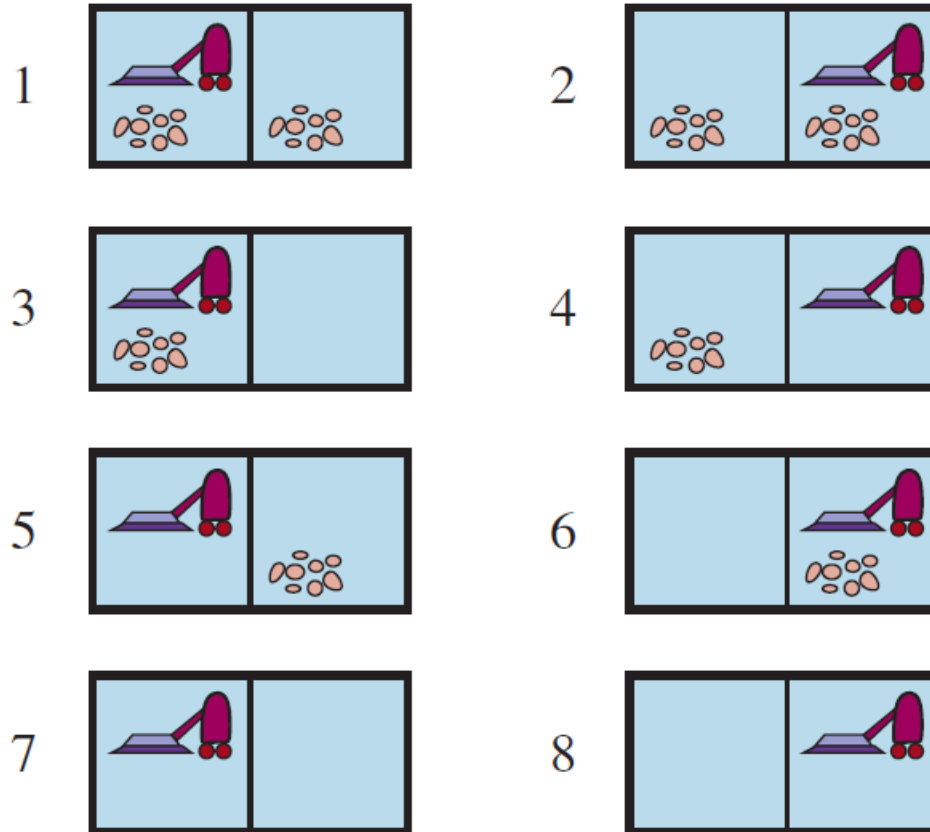
Suck:

- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square

No single sequence of actions solves the problem.

Solution still exists as a *conditional plan* (suppose at State 1):

[*Suck*, if State = 5 then
[*Right*, *Suck*] else []]



Still Solvable?

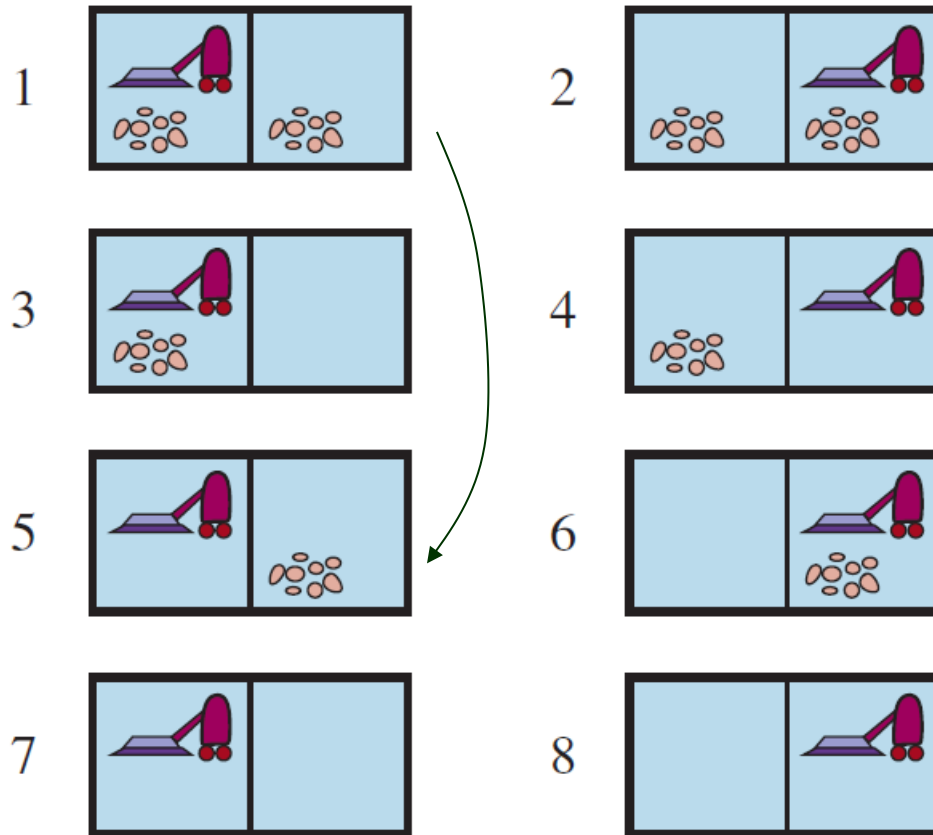
Suck:

- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square

No single sequence of actions solves the problem.

Solution still exists as a *conditional plan* (suppose at State 1):

[*Suck*, if State = 5 then
[*Right*, *Suck*] else []]



Still Solvable?

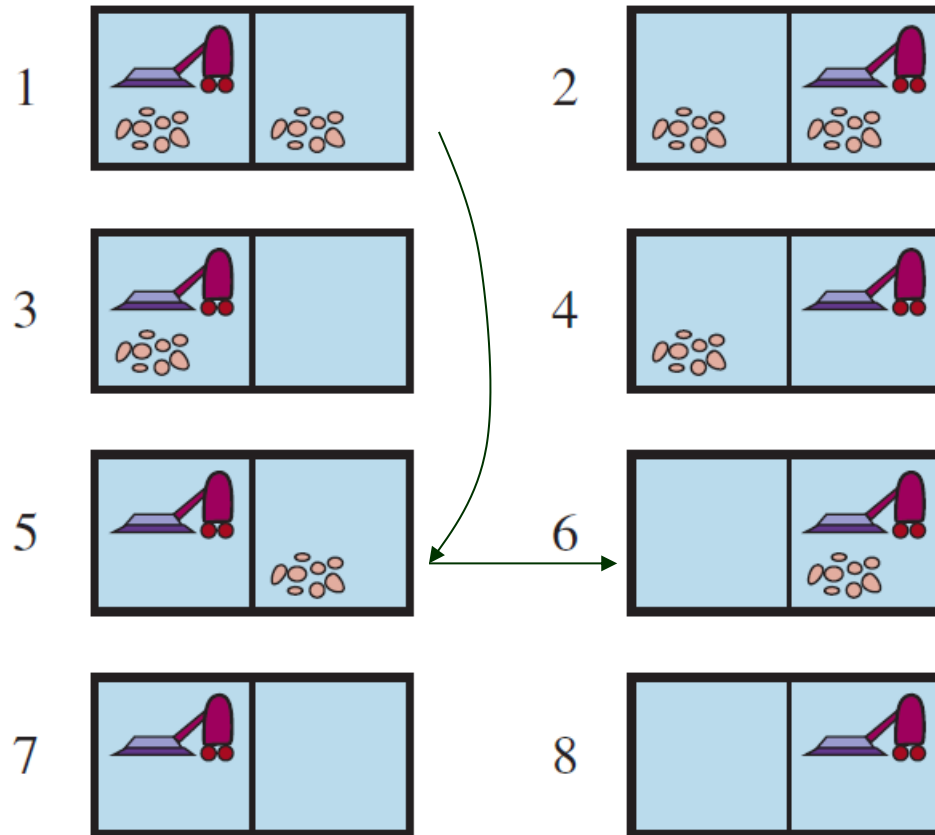
Suck:

- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square

No single sequence of actions solves the problem.

Solution still exists as a *conditional plan* (suppose at State 1):

[*Suck*, if State = 5 then
[*Right*, *Suck*] else []]



Still Solvable?

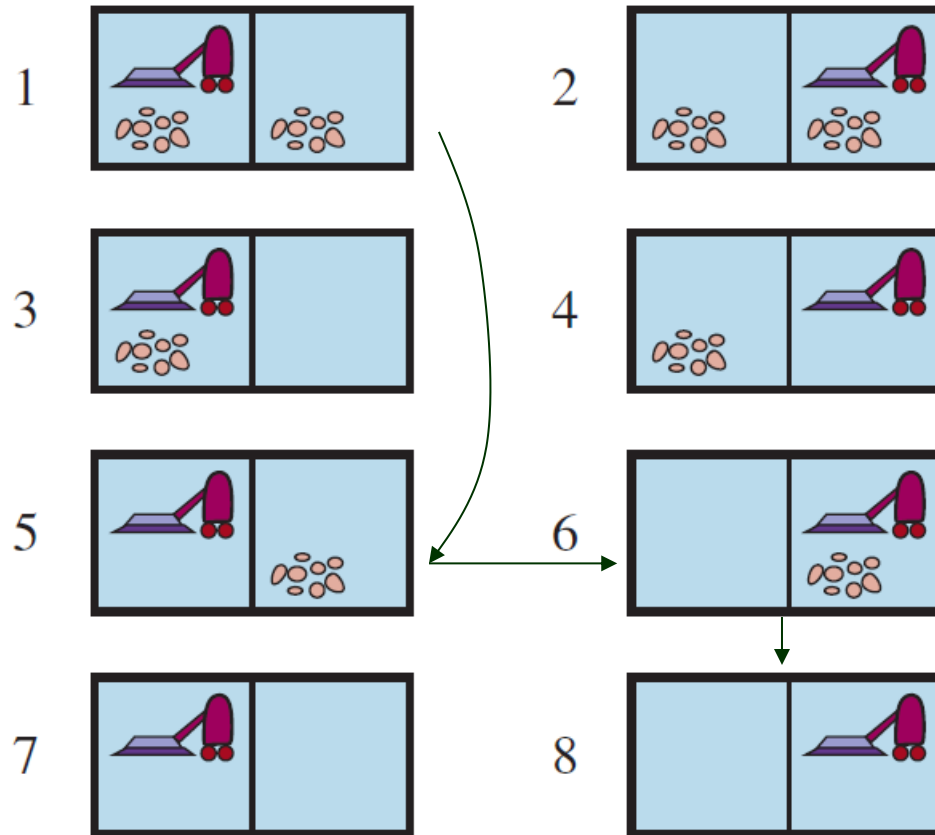
Suck:

- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square

No single sequence of actions solves the problem.

Solution still exists as a *conditional plan* (suppose at State 1):

[*Suck*, if State = 5 then
[*Right*, *Suck*] else []]



Still Solvable?

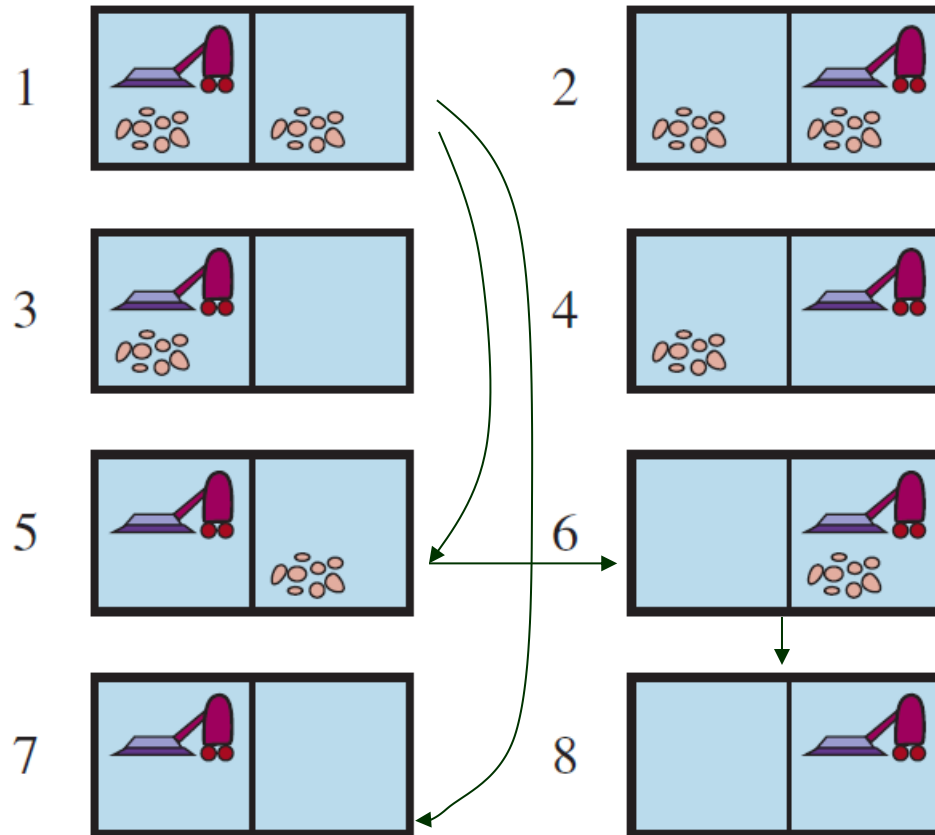
Suck:

- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square

No single sequence of actions solves the problem.

Solution still exists as a *conditional plan* (suppose at State 1):

[*Suck*, if State = 5 then
[*Right*, *Suck*] else []]



Still Solvable?

Suck:

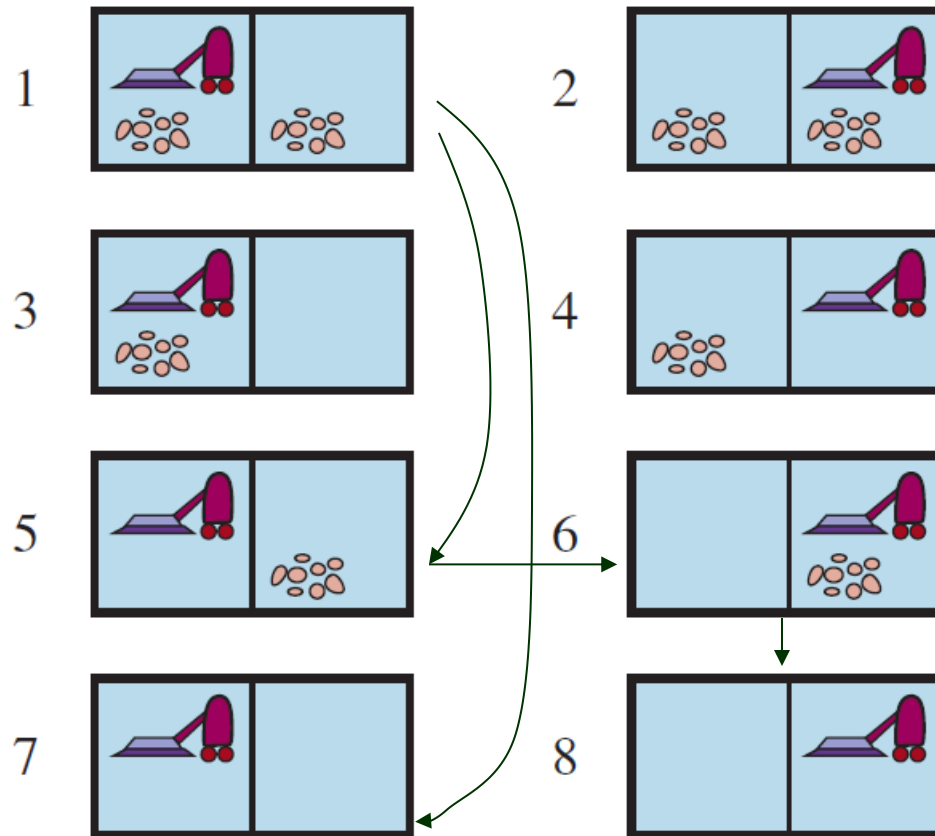
- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square

No single sequence of actions solves the problem.

Solution still exists as a *conditional plan* (suppose at State 1):

[*Suck*, if State = 5 then
[*Right*, *Suck*] else []]

Solution is a tree – of a different character!



Suck:

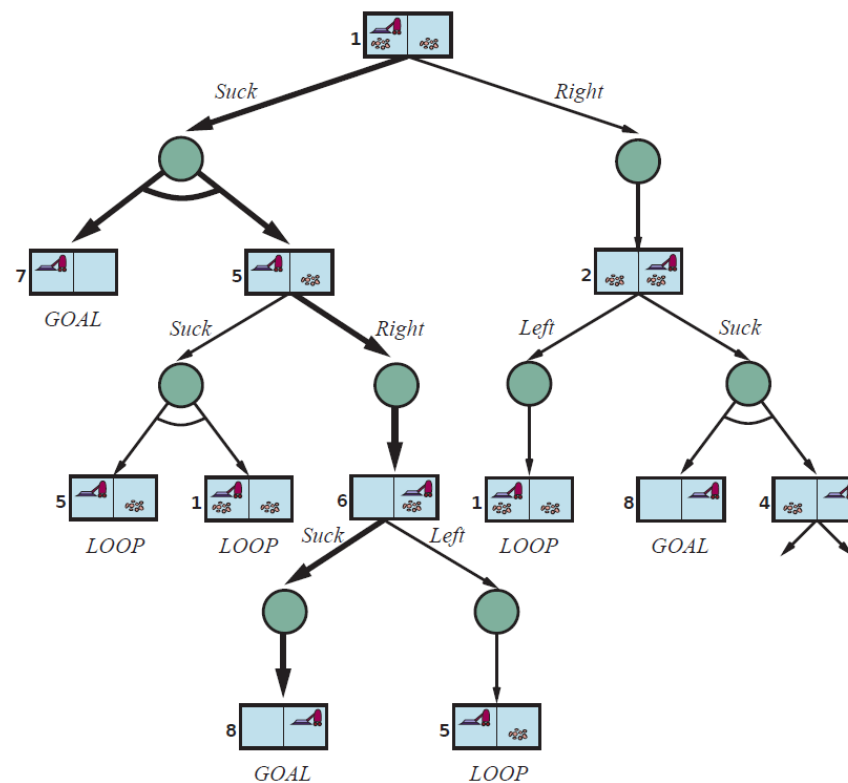
- Applied to a dirty square
 - ♦ clean the square
 - ♦ sometimes also clean up dirt in an adjacent square

AND-OR Search Tree

- Applied to a clean square
 - ♦ sometimes deposit dirt on the carpet

OR-node (*deterministic*):
the agent chooses an action.

e.g., *Left, Right, or Suck*



Suck:

- Applied to a dirty square
 - ♦ clean the square
 - ♦ sometimes also clean up dirt in an adjacent square

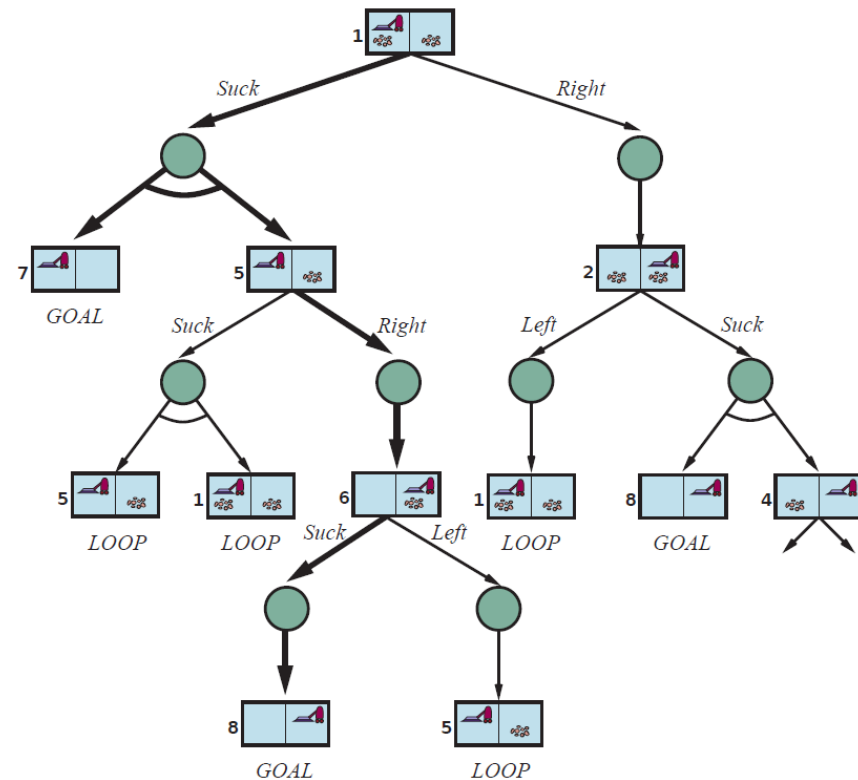
AND-OR Search Tree

- Applied to a clean square
 - ♦ sometimes deposit dirt on the carpet

OR-node (*deterministic*):

the agent chooses an action.

e.g., *Left*, *Right*, or *Suck*



AND-node (*non-deterministic*): the environment “chooses” to have an outcome for each action.

e.g., *Suck* in state 1 results in the belief state {5, 7}.

Suck:

- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square

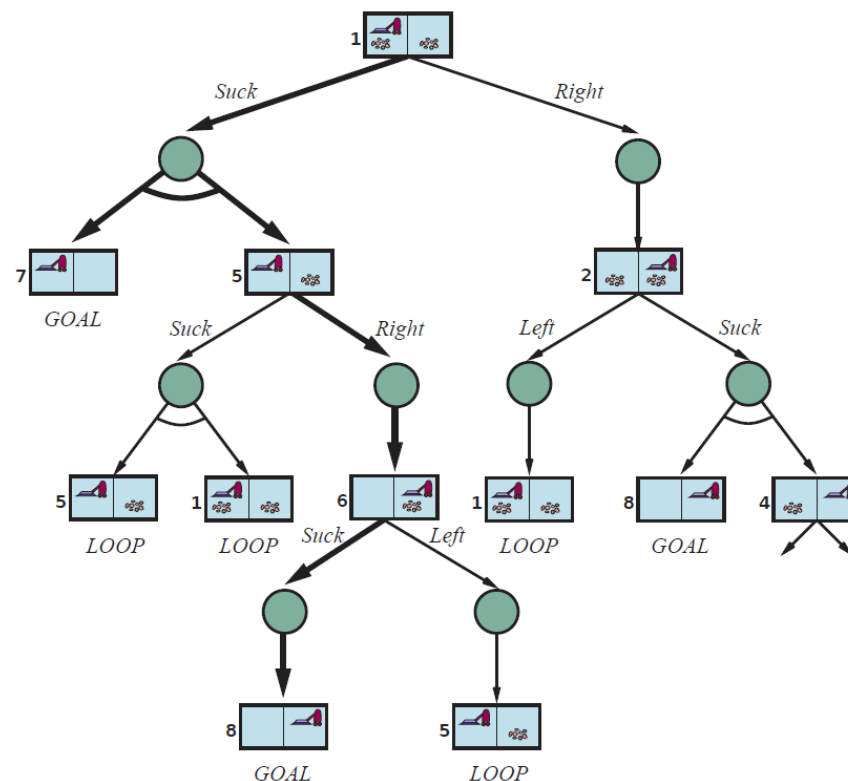
AND-OR Search Tree

- Applied to a clean square
 - ◆ sometimes deposit dirt on the carpet

OR-node (*deterministic*):

the agent chooses an action.

e.g., *Left*, *Right*, or *Suck*

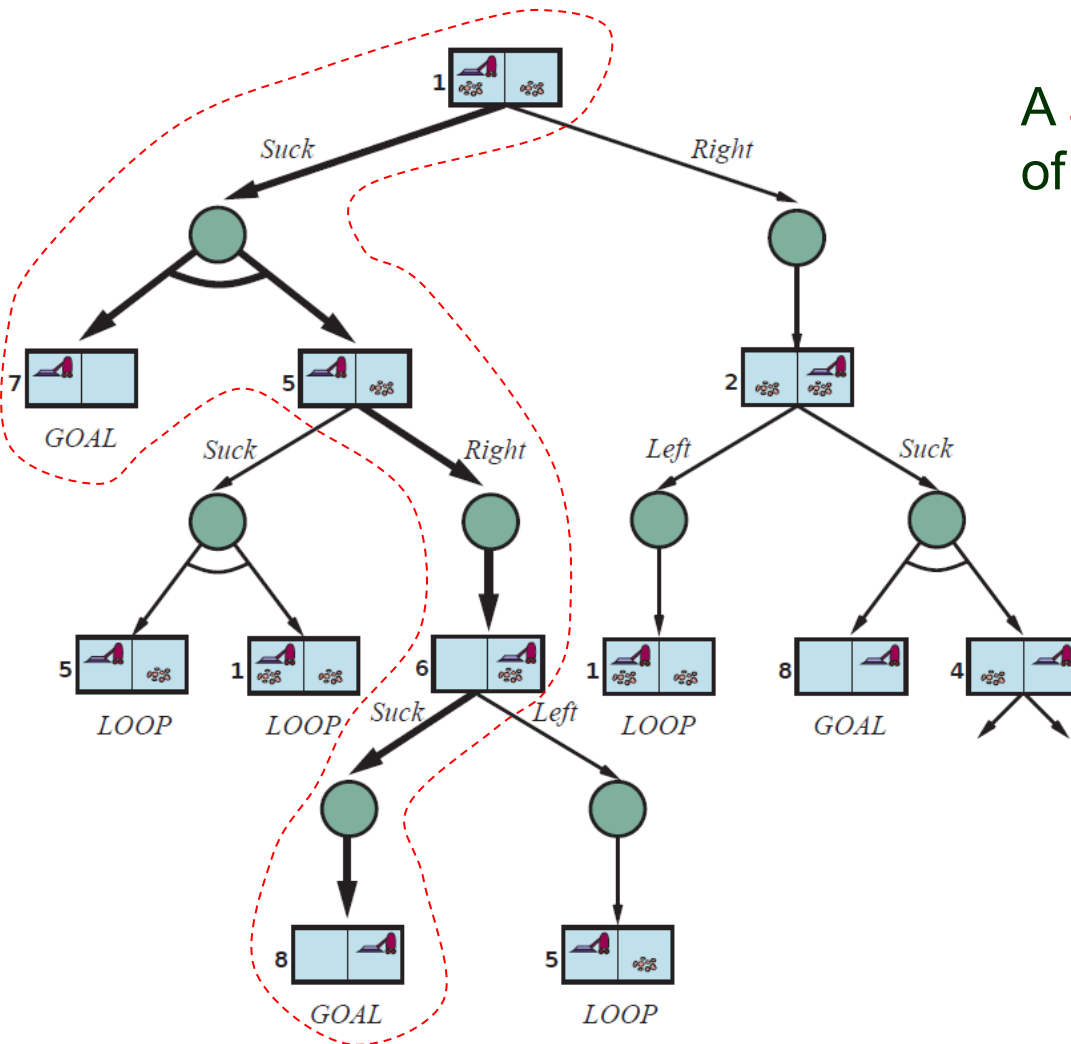


AND-node (*non-deterministic*): the environment “chooses” to have an outcome for each action.

e.g., *Suck* in state 1 results in the belief state {5, 7}.

OR- and AND-nodes alternate in the tree.

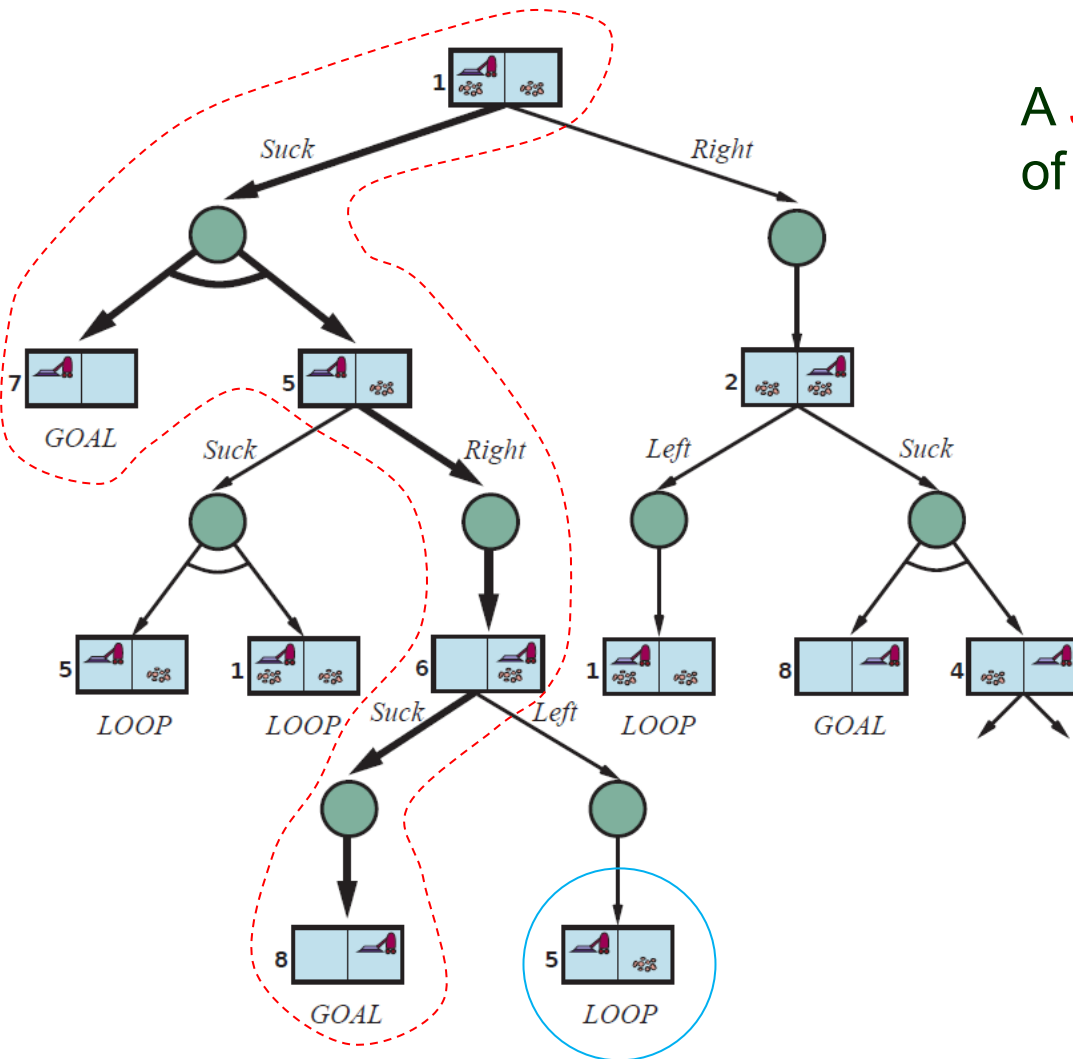
Example of a (Partial) Tree



A *solution* is a connected portion of the AND-OR tree such that

- ♦ its root is the tree's root;
- ♦ every OR node has exactly one child (i.e., one of the actions);
- ♦ every AND node has all children (possible outcomes) from the corresponding action;
- ♦ all the leaves are goal nodes.

Example of a (Partial) Tree



A *solution* is a connected portion of the AND-OR tree such that

- ♦ its root is the tree's root;
- ♦ every OR node has exactly one child (i.e., one of the actions);
- ♦ every AND node has all children (possible outcomes) from the corresponding action;
- ♦ all the leaves are goal nodes.

DFS Implementation of AND-OR Tree Search

function AND-OR-SEARCH(*problem*) **returns** a conditional plan, or *failure*

return OR-SEARCH(*problem*, *problem*.INITIAL, []) $\{a_1, a_2, \dots, a_m\}$

function OR-SEARCH(*problem, state, path*) **returns** *a conditional plan, or failure*

if *problem.IS-GOAL*(*state*) **then return** the empty plan

if IS-CYCLE($path$) then return *failure* // ignore a solution with a cycle. such a solution would

```

for each action in problem.ACTIONS(state) do // imply the existence of a non-cyclic solution
    plan ← AND-SEARCH(problem, RESULTS(state, action), [state] + path) // which can
    // be found.

```

```

if  $plan \neq failure$  then return  $[action] + plan$  // one action eventually leads
return failure // to a solution; terminates successfully.

```

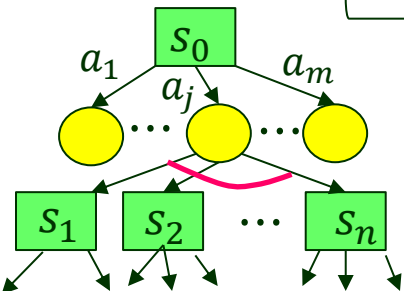
return *failure*

function AND-SEARCH(*problem, states, path*) **returns** a conditional plan, or failure

for each s_i in *states* do
$$plan_i \leftarrow \text{OR-SEARCH}(problem, s_i, path)$$

```
if  $plan_i = failure$  then return  $failure$  // one state has no solution; fails to solve the problem.
```

return [if s_1 then $plan_1$ else if s_2 then $plan_2$ else ... if s_{n-1} then $plan_{n-1}$ else $plan_n$]



Recursion Tree

